

Data Analysis Algorithm

Week-3

Task

Dhusyanth R

CH.SC.U4CSE24161

1.) Merge Sort:

Code:

```
1  #include <stdio.h>
2
3  void merge(int arr[], int left, int mid, int right)
4      int n1 = mid - left + 1;
5      int n2 = right - mid;
6      int leftArr[n1], rightArr[n2];
7
8      for (int i = 0; i < n1; i++)
9          leftArr[i] = arr[left + i];
10     for (int j = 0; j < n2; j++)
11         rightArr[j] = arr[mid + 1 + j];
12     int i = 0, j = 0, k = left;
13     while (i < n1 && j < n2) {
14         if (leftArr[i] <= rightArr[j]) {
15             arr[k++] = leftArr[i++];
16         } else {
17             arr[k++] = rightArr[j++];
18         }
19     }
20
21     while (i < n1) arr[k++] = leftArr[i++];
22     while (j < n2) arr[k++] = rightArr[j++];
23 }
24
25 void mergeSort(int arr[], int left, int right) {
26     if (left < right) {
27         int mid = left + (right - left) / 2;
28
29         mergeSort(arr, left, mid);
30         mergeSort(arr, mid + 1, right);
31         merge(arr, left, mid, right);
32     }
33 }
```

```

35 void printArray(int arr[], int size) {
36     for (int i = 0; i < size; i++)
37         printf("%d ", arr[i]);
38     printf("\n");
39 }
40
41 int main() {
42     int arr[] = {70, 50, 30, 10, 20, 40, 60};
43     int size = sizeof(arr) / sizeof(arr[0]);
44
45     printf("Original array: ");
46     printArray(arr, size);
47
48     mergeSort(arr, 0, size - 1);
49
50     printf("Sorted array: ");
51     printArray(arr, size);
52
53     return 0;
54 }

```

Output:

```

Original array: 70 50 30 10 20 40 60
Sorted array: 10 20 30 40 50 60 70
-----
Process exited after 0.0377 seconds with return value 0
Press any key to continue . . . |

```

2.) Quick Sort:

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  void swap(int* a, int* b) {
6      int temp = *a;
7      *a = *b;
8      *b = temp;
9  }
10
11 int partition(int arr[], int low, int high) {
12     srand(time(0));
13     int randomIndex = low + rand() % (high - low + 1);
14     swap(&arr[randomIndex], &arr[high]);
15
16     int pivot = arr[high];
17     int i = (low - 1);
18
19     for (int j = low; j <= high - 1; j++) {
20         if (arr[j] <= pivot) {
21             i++;
22             swap(&arr[i], &arr[j]);
23         }
24     }
25     swap(&arr[i + 1], &arr[high]);
26     return (i + 1);
27 }
```

```

27 - /
28
29 void quickSort(int arr[], int low, int high) {
30     if (low < high) {
31         int pi = partition(arr, low, high);
32         quickSort(arr, low, pi - 1);
33         quickSort(arr, pi + 1, high);
34     }
35 }
36
37 void printArray(int arr[], int size) {
38     for (int i = 0; i < size; i++) {
39         printf("%d ", arr[i]);
40     }
41     printf("\n");
42 }
43
44 int main() {
45     int arr[] = {12, 17, 6, 25, 1, 5};
46     int n = sizeof(arr) / sizeof(arr[0]);
47
48     printf("Unsorted array: ")

```

Output:

```

Unsorted array: 12 17 6 25 1 5
Sorted array: 1 5 6 12 17 25

-----
Process exited after 0.03858 seconds with return value 0
Press any key to continue . . . |

```